SN-LPA: Size Normalized Label Propagation Algorithm

Lakshay (2023UCA1838)
Dept. of Computer Science and Engineering
Netaji Subhas University of Technology (NSUT)
New Delhi, India

Aman Tomar (2023UCA1818)
Dept. of Computer Science and Engineering
Netaji Subhas University of Technology (NSUT)
New Delhi, India

Under the supervision of Dr. Vandana Bhatia
Dept. of Computer Science and Engineering
Netaji Subhas University of Technology (NSUT)
New Delhi, India

Abstract—Label propagation finds communities in near-linear time, but it is unstable and blind to resolution. TPOC-LPA (Kong et al., 2026) is a recent three-stage variant that seeds communities from node importance, evolves labels by an attractiveness score, and filters labels to allow overlap. We re-implement it and find that its Stage 2 score sums neighbour attractiveness with no size normalisation, so a growing community is rewarded for size alone: on a dense graph (email-Eu) it collapses to one community (NMI 0.00), and on sparse graphs it under-merges. We propose SN-LPA (Size Normalized Label Propagation Algorithm): six modifications, all optional and off by default so that the default behaviour is the published algorithm: a triangle-based similarity, a best-iterate guard, a size-corrected gain, a label-free granularity rule that chooses between the two regimes, a merge pass that stops at the minimum of the map equation, and boundary-node moves scored by the Synwalk objective. We compare SN-LPA with Fastgreedy, LPA, Louvain and Walktrap, re-run on identical graphs, plus Infomap as an extra. It has the best mean rank on 11 real networks and the highest mean NMI on six LFR settings (0.9941 against 0.9905 for Infomap), but it is far behind Infomap on SNAP-Youtube. On overlapping LFR graphs it beats SLPA on four of six families and loses on the two largest, and on the extended modularity EQ it wins on 5 of 12 real networks against SLPA. We also report the ideas that failed and the limits we could not remove.

Index Terms—community detection, label propagation, overlapping communities, map equation, resolution limit, node importance

I. Introduction

A. Background and Motivation

Many real systems are networks: people and their friendships, authors and their co-authorships, proteins and their interactions, web pages and their links. A recurring feature of such networks is community structure: the nodes fall into groups that are densely connected inside and sparsely connected to the rest [2], [3]. Finding these groups, called community detection, helps to understand how a network is organised, to predict missing links and attributes, and to summarise a graph that is too large to inspect. The task is hard to define exactly, so a large number of methods exist: they optimise modularity [4], simulate random walks [13], [14], or let nodes exchange labels [5].

Label propagation algorithms (LPA) are among the cheapest of these. Every node starts with its own label and repeatedly adopts the label that is most common among its neighbours; connected groups converge to a shared label. One pass costs $O(m)$ for a graph with m edges, there is no parameter to set, and the number of communities need not be known. This is why LPA and its descendants are used on graphs with millions of edges [6], [7]. The price is quality. Plain LPA is non-deterministic because ties are broken at random, it can converge to very different partitions on different runs, and on some graphs a single label takes over the whole network [5].

B. From Label Propagation to TPOC-LPA

A long line of work therefore adds structure to the propagation rule. Some methods replace the majority vote by a modularity gain [8] and merge communities afterwards [9]. Others fix the order in which nodes are updated, or weight the votes by how important a node is in the graph. A second need is overlap: a person belongs to several social circles, a protein to several complexes, and a method that assigns each node to exactly one group cannot express this. Overlapping variants let a node keep several labels [22], [23].

TPOC-LPA [1] is a recent method that combines these ideas in three stages. Stage 1 seeds communities around nodes of high importance, growing each by a tightness–sparsity test. Stage 2 lets every node adopt the community that attracts it most, where the attraction combines neighbour importance with a neighbourhood-similarity term. Stage 3 filters the labels of nodes that touch several communities, so that a node keeps more than one label only if its affiliation to each is strong enough. The paper

reports the best average NMI among seven methods on eight real networks, the best NMI on all six of its LFR benchmarks, and the best extended modularity on most of 12 real networks for overlap. We took it as our starting point.

C. What We Observed

We re-implemented the algorithm from the paper, checked every formula against the worked examples in it, and then tested it beyond the graphs it reports. Two behaviours stand out. First, on a dense graph (email-Eu-core: 986 nodes, 42 departments, average degree 33) Stage 1 gives 335 communities at NMI 0.54, and Stage 2 then merges all of them into a single community, so NMI drops to 0.00. On LFR graphs with a realised mixing parameter of $\mu = 0.7$ the NMI averages 0.05. Second, on sparse graphs the opposite happens: on Cora Stage 2 leaves 214 communities for 7 classes. The cause is in the Stage 2 score (Eq. (14) below): it sums neighbour attractiveness with no term that depends on community size, so a community that grows is rewarded for its size alone.

D. Research Gap

From this reading and from our experiments we identify the following gaps.

1) No analysis of the size effect in Stage 2. Label propagation with an unnormalised score is known to be unstable; modularity-specialised variants fix it with a null-model term [8]. TPOC-LPA does not include such a term, and its behaviour on dense graphs and on graphs with high mixing was not studied; the benchmark graphs in the paper are those where the rule works.
2) Resolution is not controlled. The propagation stops at a local fixed point with no check of the scale of the communities. Merge steps that exist in related work use modularity, whose resolution limit depends on the size of the whole network [16], and we show that this can destroy a good partition (Sec. IV).
3) Missing strong baselines and reproducibility. The comparison in [1] omits methods based on the map equation, and six of its baselines (NSA, SimCMR, WLC, UMSTMO, GREESE, MDPA) have no public implementation that we could find. Published numbers for them cannot be checked, and the choice of the NMI normalisation is not stated.
4) The overlap rate is not calibrated. The affiliation threshold β is a fixed parameter, and nothing relates it to how many nodes of a given graph should be overlapping.
5) Where a change helps and where it hurts is rarely reported. Improvements to label propagation are usually shown on the graphs they were designed for. A change that helps on one family of graphs can hurt on another, and this needs a protocol with development and held-out graphs.

E. Contributions

1) Diagnosis. We re-implement TPOC-LPA and show that its Stage 2 rule is the γ=0 end of a size-corrected family, which explains both the collapse on dense graphs and the under-merging on sparse ones (Sec. III-D).
2) SN-LPA, a pipeline of six optional modifications (M1–M6), with the published behaviour as the default: triangle similarity, best-iterate guard, size-corrected gain, a label-free granularity rule that chooses between the two regimes, a merge pass stopped by the map equation, and boundary moves scored by the Synwalk objective with a regularised acceptance gate. The ideas come from existing work; the specific combination, the rule and the gate are ours (Sec. III-E). We give the algorithm in pseudocode (Algorithms 1–4).
3) A like-for-like comparison. We re-run the baselines of [1] that have public code, plus Infomap, on identical graphs with identical metric code, and report wins and losses, including the results that do not favour us (Sec. IV).
4) Negative results and limits. We report the ideas that failed and the gaps we could not close (Sec. V).

F. Organisation

Section II reviews related work. Section III describes the datasets, the evaluation metrics, the baseline, its problems and the proposed architecture. Section IV presents the results, Section V discusses them, and Section VI concludes.

II. Related Work and Literature Review

A. Community Detection: Objectives and Families

Girvan and Newman [2] popularised community detection with a divisive method and with the football and karate networks as test cases; Fortunato [3] surveys the field. Methods differ by what they optimise. Modularity-based methods maximise the difference between the number of edges inside communities and its expectation in a degree-preserving random graph [4]: Fastgreedy [10] merges communities greedily, and Louvain [11] alternates local moves with aggregation. Leiden [12] refines Louvain to guarantee connected communities (we did not run it). Random-walk methods use the idea that a walker stays inside a community for a long time: Walktrap [13] clusters nodes by walk distances, and the map equation [14] minimises the code length of a walker's path. Lancichinetti and Fortunato [15] compare many of these methods on benchmark graphs.

B. Label Propagation and its Variants

Raghavan et al. [5] introduced the majority-vote rule. Leung et al. [6] analysed its behaviour on large networks

and proposed ways to stabilise it. Traag and Šubelj [7] make label propagation faster by processing only nodes whose neighbourhood changed. LPAm [8] derives a label update from modularity: the gain of joining community c is the number of links into c minus the number expected under the configuration model. LPAm can get stuck in poor local optima, and LPAm+ [9] adds a multistep greedy merge of communities to escape them, in the spirit of [10]. Li et al. [20] weight propagation by motifs (triangles), which motivates our similarity.

C. Overlapping Community Detection

The clique percolation method of Palla et al. [21] made overlap a central topic. In label propagation, COPRA [23] lets each node hold several labels with belonging coefficients, and SLPA [22] stores the labels a node has received over T rounds and keeps those whose relative frequency exceeds a threshold r; it is the best known overlapping label-propagation baseline and we use it. Ego-splitting [24] clusters a graph whose nodes are the "personas" of each node. TPOC-LPA [1] filters labels by a weighted affiliation score (WAS) and keeps those above a threshold times the node's mean score.

D. Resolution Limit, Map Equation and Random-Walk Objectives

Fortunato and Barthélemy [16] showed that modularity cannot resolve communities smaller than a scale that depends on the total number of links in the network. The map equation [14] avoids this: Kawamoto and Rosvall [17] derive that its resolution limit is set by the number of links between modules instead. On sparse networks the map equation tends to over-fit, which regularised versions address with a prior on unobserved links [18]. Synwalk [19] fits a random-walk model to a partition and is reported to outperform Infomap on networks with high mixing. These three results are what our merge stop (M5) and boundary moves (M6) rest on.

E. TPOC-LPA and Where We Stand

Table I summarises how the methods above relate to this work. TPOC-LPA [1] combines node importance, a CTS-gated pre-selection and WAS filtering, but it has no size correction, no resolution control and no calibrated overlap rate. We keep all of its stages and replace or extend the middle of the pipeline.

TABLE I
Related methods and their role in this work.

Method	Key idea	Used here as
LPA [5]	Majority label vote	Baseline (run)
LPAm / LPAm+ [8], [9]	Modularity gain; greedy merge	Size-corrected gain (M3); merge order (M5)
Li et al. [20]	Triangle-weighted propagation	Motivation for M1
Louvain, Fastgreedy [10], [11]	Modularity optimisation	Baselines (run)
Walktrap [13]	Walk distances	Baseline (run)
Infomap [14], [17], [18]	Map equation; resolution limit; regularisation	Extra baseline; merge stop (M5); gate (M6)
Synwalk [19]	Walk-model objective J	Move score (M6)
SLPA [22]	Label memory, threshold r	Overlap baseline (run)
TPOC-LPA [1]	NI seeds, CTS, WAS	Baseline (re-implemented)

Sourcing note. The cited methods support the ideas we use; they do not show that the ideas work inside TPOC-LPA. That evidence is our experiments, including the ones that failed. We read several of the cited papers only in part (the abstracts and the sections defining the objectives we use), and we had no access to the full text of [20].

III. Methodology

A. Datasets

We use four groups of graphs. All graphs are treated as undirected and unweighted. (We found that the Karate graph shipped with networkx carries edge weights; we removed them, see Sec. V.)

Real networks with ground truth. Table II lists them: Karate [30], Dolphin (four groups reconstructed from the original study [31]), Football [2] (conferences), Polbooks (political leaning), Polblogs [32], email-Eu-core (departments, from [33]), Cora and Citeseer [34] (paper classes), and three SNAP graphs with ground-truth communities [29]. The SNAP graphs are our own induced subgraphs, 2.6–3.9× larger than the subsets used in [1], with the first listed community as the label of each node; absolute values are therefore not comparable to the paper, only across our columns. Karate to Citeseer use the largest connected component.

TABLE II
Real networks used in the experiments.

Network	n	m	Classes	Truth
Karate	34	78	2	two factions
Dolphin	62	159	4	four groups (reconstructed)
Football	115	613	12	conferences
Polbooks	105	441	3	political leaning
Polblogs	1,222	16,714	2	political leaning
email-Eu	986	16,064	42	departments
Cora	2,485	5,069	7	paper classes
Citeseer	2,110	3,668	6	paper classes
SNAP-Amazon	16,716	48,739	1,399	first community
SNAP-Youtube	39,841	224,235	4,447	first community
SNAP-DBLP	93,432	335,520	4,923	first community

LFR benchmarks (disjoint). We use the generator of Lancichinetti et al. [25] with the six settings of [1] (Table III): average degree 20, power-law exponents $\tau_1 = 3$, $\tau_2 = 1.5$, mixing $\mu \in \{0.1, 0.6\}$. The generator in networkx realises a mixing far above its nominal value (nominal 0.6 gives about 0.84), so we search the nominal value until the realised mixing equals the target. For $n = 1000$ the minimum community size is 15 instead of 10, since 10 is infeasible with degree 20 in this generator. Five graphs (seeds 1–5) are generated per setting.

TABLE III
LFR settings for disjoint communities.

Name	n	$k_{\max}$	$c_{\min}$–$c_{\max}$	μ
N1 / N2	1000	50	15–50	0.1 / 0.6
N3 / N4	5000	50	20–100	0.1 / 0.6
N5 / N6	10000	100	20–200	0.1 / 0.6

LFR benchmarks (overlapping). The six families R1–R6 of Table 7 in [1]: $n \in \{1000, 5000, 10000\}$, $\mu \in \{0.1, 0.3\}$, average degree 10 (20 for $n = 10000$), and $om \in \{2, \ldots, 8\}$ memberships for the overlapping nodes ($on = 100, 500, 2000$ of them); 42 graphs per seed, two seeds, from the original generator of Lancichinetti et al.

Networks for EQ. Karate, Dolphin, Lesmis, Jazz, Cora (2708 nodes), Power, Facebook, CA-GrQc, LastFM-Asia, CA-HepTh, PGP and Deezer. Eleven of the twelve match the node and edge counts in [1] (CA-GrQc and CA-HepTh differ by two nodes).

B. Evaluation Metrics

We report two kinds of metrics. External metrics compare a detected result X with the ground truth Y and measure agreement; internal metrics (modularity, EQ) need no ground truth and measure how well the result fits the graph's own structure. The two can disagree: a partition with higher modularity can match the truth worse, as we show on Karate. Throughout, N is the number of nodes and m the number of edges; all scores are better when higher. Table IV gives an overview, and the rest of this subsection explains each metric and gives its definition.

TABLE IV
Evaluation metrics.

Metric	Result type	Needs truth	Range
NMI	disjoint	yes	$[0, 1]$
Modularity Q	disjoint	no	≤ 1
LFK-NMI	overlapping	yes	$[0, 1]$
$\text{NMI}_{\max}$	overlapping	yes	$[0, 1]$
Omega index	overlapping	yes	≤ 1
EQ	overlapping	no	≤ 1

1) Normalised Mutual Information (NMI)

NMI asks how much knowing a node's detected community tells us about its true community. It is the mutual information of the two labelings, divided by their entropies so that the score lies in $[0, 1]$: 1 for identical partitions and close to 0 for unrelated ones. It ignores how communities are named, and the two partitions need not have the same number of communities. With n_k the size of community k,

$$\text{NMI}(X, Y) = \frac{2\, I(X;Y)}{H(X) + H(Y)}, \tag{1}$$

where $H(X) = -\sum_k \frac{n_k}{N} \log \frac{n_k}{N}$ and $I(X;Y) = H(X) + H(Y) - H(X,Y)$. We use this arithmetic normalisation in every column of our tables (see Sec. IV-A).

2) Modularity (Q)

Modularity compares the fraction of edges inside communities with the fraction expected if the same nodes were wired at random while keeping their degrees. A high value means many more internal edges than chance would give. It needs no ground truth, but it has a resolution limit [16], so a high Q does not guarantee that communities match the truth.

$$Q = \frac{1}{2m} \sum_{u,w} \left(A_{uw} - \frac{k_u k_w}{2m} \right) \delta(c_u, c_w), \tag{2}$$

where A is the adjacency matrix, k_u the degree of u, and $\delta(c_u, c_w) = 1$ if u and w share a community. We compute Q on the unweighted graph.

3) LFK-NMI and $\text{NMI}_{\max}$ (overlapping covers)

NMI assumes that every node has one label. For covers, Lancichinetti et al. [26] treat each community X_k as a yes/no variable ("is this node in X_k?") with entropy

$$H(X_k) = h(p_k) + h(1 - p_k), \qquad p_k = \frac{|X_k|}{N}, \tag{3}$$

where $h(p) = -p \log_2 p$. How much the other cover Y tells us about X_k is measured through its best-matching community,

$$H(X_k|Y) = \min_l H(X_k|Y_l), \tag{4}$$

taken over those Y_l that satisfy the constraint of [26] (a match counts only if it is better than chance). LFK-NMI averages this over the communities, each normalised by its own entropy:

$$\tilde{H}(X|Y) = \frac{1}{|X|} \sum_k \frac{H(X_k|Y)}{H(X_k)}, \tag{5}$$

$$\text{NMI}_{\text{LFK}} = 1 - \tfrac{1}{2}\left[\tilde{H}(X|Y) + \tilde{H}(Y|X)\right]. \tag{6}$$

The variant $\text{NMI}_{\max}$ sums the entropies instead and divides by the larger one, which is stricter:

$$I(X:Y) = \tfrac{1}{2}\left[H(X) + H(Y) - H(X|Y) - H(Y|X)\right], \tag{7}$$

$$\text{NMI}_{\max} = \frac{I(X:Y)}{\max\{H(X), H(Y)\}}, \tag{8}$$

with $H(X) = \sum_k H(X_k)$ and $H(X|Y) = \sum_k H(X_k|Y)$.

4) Omega index

Omega is a pair-counting measure. For every pair of nodes it counts in how many communities the pair appears

together, and checks whether the detected and the true cover give the same count. It generalises the Rand index to overlapping covers [27]; the correction for chance makes 0 mean "no better than random". Let $t_X(p)$ be the number of communities of X that contain both nodes of pair p, and $N_p = \binom{N}{2}$. Then

$$\Omega = \frac{\Omega_u - \Omega_e}{1 - \Omega_e}, \qquad \Omega_u = \frac{1}{N_p} \sum_p \mathbf{1}\big[t_X(p) = t_Y(p)\big], \quad (9)$$

and $\Omega_e = \sum_j A_j B_j / N_p^2$ is the agreement expected by chance, where A_j (B_j) is the number of pairs with $t_X = j$ ($t_Y = j$). For partitions Ω equals the adjusted Rand index; we checked this numerically.

5) Extended modularity (EQ)

EQ extends modularity to covers [28]. A node in O_u communities contributes to each of them with weight $1/O_u$, so overlap is not counted twice. It needs no ground truth, it equals Q for partitions, and it is the measure used in [1] for overlapping results on real networks.

$$\mathrm{EQ} = \frac{1}{2m} \sum_c \sum_{u,w \in c} \frac{1}{O_u O_w} \Big(A_{uw} - \frac{k_u k_w}{2m} \Big). \quad (10)$$

Evaluated directly this costs $O(\sum_c |c|^2)$. We use the equivalent form

$$\mathrm{EQ} = \frac{1}{2m} \sum_{(u,w) \in E} \frac{2\, s_{uw}}{O_u O_w} - \frac{1}{2m} \sum_c \frac{S_c^2}{2m}, \quad (11)$$

with s_{uw} the number of communities shared by u and w, and $S_c = \sum_{u \in c} k_u / O_u$. It costs $O(m)$, and we verified it against the direct sum.

C. Baseline: TPOC-LPA

Fig. 1 shows the architecture and Algorithm 1 the pseudocode, which follows Algorithm 1 of [1].

Node importance. For node i with degree d_i and c_i edges among its neighbours, $f_i = d_i\,(1 + c_i / \sum_k c_k)$ is min–max scaled to $e_i = 0.5 + 0.5\,\frac{f_i - f_{\min}}{f_{\max} - f_{\min}}$, and

$$NI(i) = e_i + \frac{1}{|N(i)|} \sum_{j \in N(i)} e_j. \quad (12)$$

Stage 1: pre-selection. The unlabelled node with the largest NI becomes a core. Its unlabelled neighbours are visited in decreasing NI; a neighbour joins if the tightness–sparsity value of the community, $\mathrm{CTS} = c_t/(1 + c_s)$ with $c_t = e_{in}/n_c$ and $c_s = e_{out}/(e_{in} + e_{out})$, does not fall below α times its previous value. This repeats until every node has a label.

Stage 2: evolution. The attractiveness of an edge is

$$A(i,j) = \frac{NI(i)\, NI(j)}{(2 - \mathrm{Sim}(i,j))^2}, \quad \mathrm{Sim}(i,j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}. \quad (13)$$

Algorithm 1 TPOC-LPA (baseline, after [1])

Input: $G = (V, E)$, α, β, max_iter
Output: Overlapping community set $Communities$

// Step 1: community pre-selection
for all $i \in V$ do compute $NI(i)$ by Eq. (12)
end for
$U \leftarrow V$
while $U \neq \emptyset$ do
  $u \leftarrow \arg\max_{x \in U} NI(x)$; $label(u) \leftarrow u$; $U \leftarrow U \setminus \{u\}$
  $s \leftarrow \{u\}$; $CTS_{prev} \leftarrow 0$
  for all neighbours $v \in U$ of u in descending $NI(v)$ do
    compute CTS_{curr} of $s \cup \{v\}$
    if $CTS_{curr} \geq \alpha \cdot CTS_{prev}$ then
      $s \leftarrow s \cup \{v\}$; $label(v) \leftarrow u$
      $U \leftarrow U \setminus \{v\}$; $CTS_{prev} \leftarrow CTS_{curr}$
    end if
  end for
end while
// Step 2: community evolution
$iter \leftarrow 0$
repeat
  for all $i \in V$ in descending $NI(i)$ do
    $RC \leftarrow \{label(j) : j \in N(i)\}$
    if $|RC| = 1$ then
      $label(i) \leftarrow$ the label in RC
    else
      $label(i) \leftarrow \arg\max_{c \in RC} \sum_{j \in N(i) \cap c} A(i,j)$
    end if
  end for
  $iter \leftarrow iter + 1$
until no label changed or $iter = max_iter$
// Step 3: community label filtering
for all $i \in V$ do $Communities[i] \leftarrow \{label(i)\}$
end for
for all $i \in V$ do
  $CL \leftarrow \{label(j) : j \in N(i)\}$
  if $|CL| > 1$ then
    compute $\mathrm{WAS}(i,c)$, $c \in CL$ (Eq. (15))
    $\overline{\mathrm{WAS}} \leftarrow$ mean of these values
    $V_i \leftarrow \{c \in CL : \mathrm{WAS}(i,c) \geq \beta\, \overline{\mathrm{WAS}}\}$
    $Communities[i] \leftarrow Communities[i] \cup V_i$
  end if
end for
return $Communities$

In decreasing NI order, node i takes the community

$$c^*(i) = \arg\max_c \sum_{j \in N(i) \cap c} A(i,j), \quad (14)$$

(directly the only one if all neighbours share a label) until no label changes or max_iter is reached.

Stage 3: label filtering. The weighted affiliation of node i to a label c present among its neighbours is

$$\mathrm{WAS}(i,c) = \frac{\sum_{j \in N(i),\, \ell(j) = c} NI(j)}{\sum_{k \in N(i)} NI(k)}, \quad (15)$$

and i keeps, besides its own label, every c with $\mathrm{WAS}(i,c) \geq \beta \cdot \overline{\mathrm{WAS}}(i)$ ($\beta = 0.8$), where the bar is the mean over the labels present.

D. Problems with the Baseline

Reading Algorithm 1 against our experiments, we find the following problems.

1) Unbounded size effect (collapse). Eq. (14) adds up neighbour attractiveness. If community c has n_c

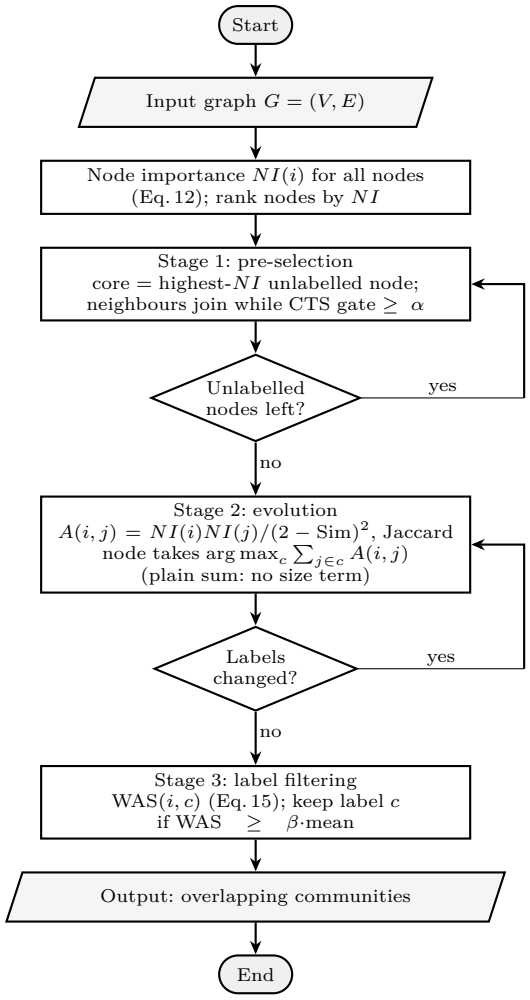


Fig. 1. Architecture of the baseline TPOC-LPA [1]. Oval = start/end, parallelogram = input/output, rectangle = process, diamond = decision.

nodes, a node adjacent to it tends to have more neighbours inside c as n_c grows, so $\sum_{j \in c} A(i,j)$ tends to grow with the size of c alone. The larger community then attracts more nodes, which makes it larger: a rich-get-richer dynamic with nothing that bounds it. On email-Eu this runs away: Stage 1 gives 335 communities at NMI 0.5415 and Stage 2 ends with one community and NMI 0.0000. On LFR graphs with realised $\mu = 0.7$ the average NMI is 0.05.

2) Under-merging on sparse graphs. When degrees are low, a node has few neighbours in any community and the sum is small, so fragments from Stage 1 do not merge. On Cora the baseline ends with 214 communities for 7 classes.
3) The rule is only one end of a family. Eq. (14) is the $\gamma = 0$ end of

$$\mathrm{gain}(c) = \sum_{j \in N(i) \cap c} A(i,j) - \gamma \frac{s_i S_c}{W}, \qquad (16)$$

where $s_i = \sum_j A(i,j)$, $S_c = \sum_{k \in c} s_k$ and $W = \sum_i s_i$. The subtracted term is the attraction expected in a random graph with the same strengths; it grows with S_c and so bounds the effect in item 1. $\gamma = 1$ is the LPAm gain [8] with importance-weighted edges. No single γ suits all graphs: a sweep of γ from 0.1 to 0.75 found none that works in both regimes.
4) The similarity can under-score tight communities. Two well-connected nodes of one community share many neighbours but have a large union, so Jaccard in Eq. (13) gives a low value for a strong link.
5) The last iterate is not always the best. Stage 2 stops at a fixed point, which can be its worst round (the collapse in item 1).
6) No control of the scale after Stage 2. Nothing merges fragments at the right scale, and the only merge steps in related work use modularity, whose resolution limit can destroy a good partition (Sec. IV).
7) Boundary nodes are not repaired. Label propagation is least reliable at community boundaries, and nothing re-examines those nodes afterwards. On LFR graphs most of the remaining error is boundary nodes; at $\mu = 0.6$ about half of the misplaced nodes have more links into another true community than into their own, so some of this error cannot be removed by any local rule.
8) The overlap rate is fixed by β. Stage 3 uses the same threshold on every graph (we return to this in Sec. V).

E. Proposed Methodology

Fig. 2 shows the architecture of the proposed pipeline, SN-LPA (Size Normalized Label Propagation Algorithm); its central change is the size-normalised Stage 2 gain (M3). Stage 1 and Stage 3 of Algorithm 1 are unchanged; Stage 2 is modified and two new steps follow it. The six modifications M1–M6 address problems 4, 5, 1–2, 3, 6 and 7 of Sec. III-D, respectively. Every option defaults to off, so the default code path is Algorithm 1. In the code they are labelled CHANGE 1, 2, 3, 4, 6 and 9. The pseudocode is given in Algorithms 2–4, in the same style as Algorithm 1.

Here Evolve(L_1, γ) denotes Stage 2 started from the Stage 1 partition L_1, with the gain of Eq. (16); $\gamma = 0$ gives Eq. (14).

Preliminaries: the map equation. Two of the modifications use the map equation [14], so we introduce it first. Imagine describing the path of a random walker on the graph with as few bits as possible. If the nodes are grouped into modules, each module gets its own short codebook for node names, and one extra code announces a move from one module to another. A partition that matches real communities keeps the walker inside a module most of the time, so the module codebooks are used often and the announcement code rarely, and the description is short. The expected description length per step is the

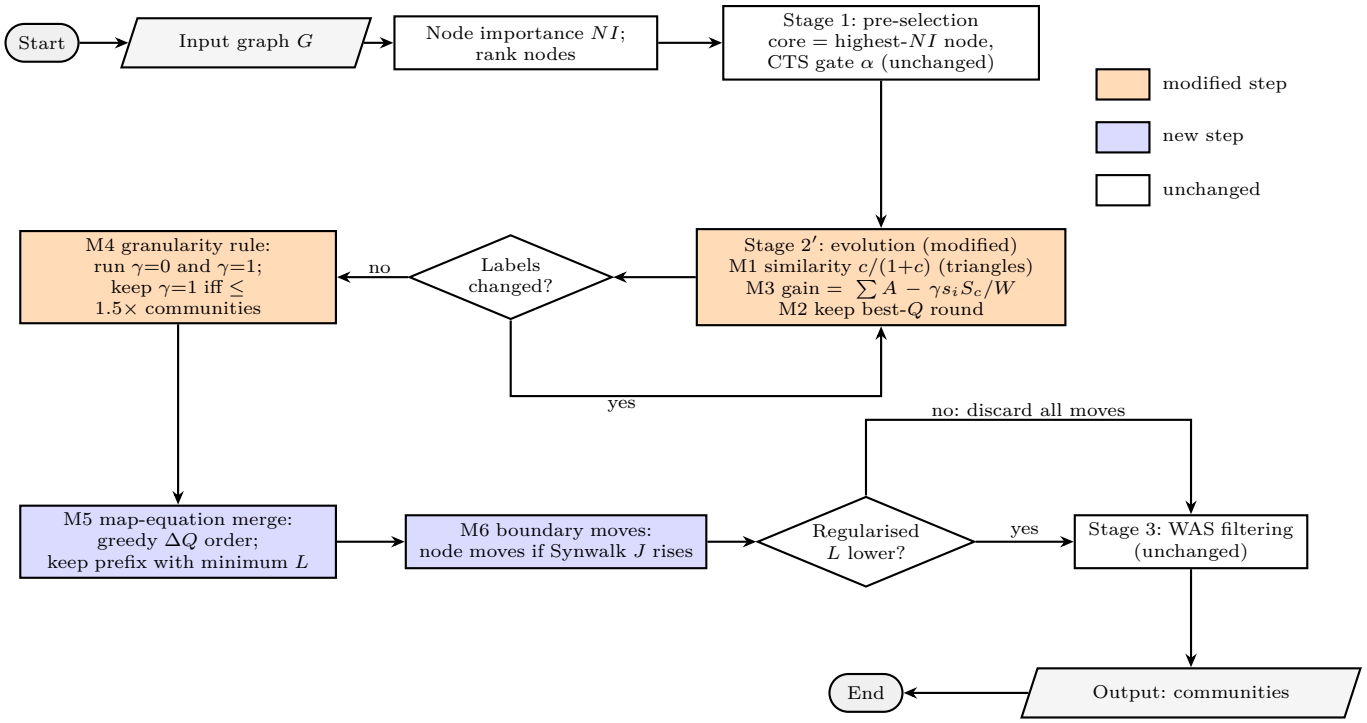


Fig. 2. Architecture of the proposed pipeline. Orange boxes modify a step of TPOC-LPA, blue boxes are new steps, white boxes are unchanged (compare Fig. 1).

Algorithm 2 Proposed pipeline (M1–M6)

Input: $G = (V, E)$, α, β, γ=1, ρ=1.5, *max_iter*, sweeps S=20
Output: Disjoint partition L, overlapping set *Communities*

// Step 1: pre-selection (unchanged)
$L_1 \leftarrow$ result of Step 1 of Algorithm 1
// Step 2′: community evolution (modified)
$\mathrm{Sim}(i,j) \leftarrow c_{ij}/(1+c_{ij})$ for all edges ▷ M1
$L^{(0)} \leftarrow \mathrm{Evolve}(L_1, \gamma{=}0)$ ▷ Eq. (14)
  keep the round with the largest Q ▷ M2
$L^{(1)} \leftarrow \mathrm{Evolve}(L_1, \gamma{=}1)$ ▷ M3, Eq. (16)
if $|L^{(1)}| \le \rho\,|L^{(0)}|$ then ▷ M4, Eq. (19)
  $L \leftarrow L^{(1)}$
else
  $L \leftarrow L^{(0)}$
end if
$L \leftarrow \mathrm{MapEqMerge}(G, L)$ ▷ M5, Alg. 3
$L \leftarrow \mathrm{SynwalkMoves}(G, NI, L, S)$ ▷ M6, Alg. 4
// Step 3: community label filtering (unchanged)
for all $i \in V$ do *Communities*$[i] \leftarrow \{L(i)\}$
end for
for all $i \in V$ with $|CL| > 1$ do
  $V_i \leftarrow \{c \in CL : \mathrm{WAS}(i,c) \ge \beta\,\overline{\mathrm{WAS}}\}$
  Communities$[i] \leftarrow$ *Communities*$[i] \cup V_i$
end for
return (L, *Communities*)

Algorithm 3 MapEqMerge(G, L) (M5)

Input: graph G, partition L
Output: partition at the lowest codelength along the merge sequence

$H \leftarrow$ max-heap of $(\Delta Q(a,b), a, b)$ over adjacent communities with $\Delta Q > 0$
$L_{best} \leftarrow L$(input) by Eq. (17); $k \leftarrow 0$; $k^* \leftarrow 0$; $seq \leftarrow [\,]$
while $H \neq \emptyset$ do
  pop (g, a, b)
  if a or b no longer exists, or not adjacent then continue
  end if
  if $g \neq \Delta Q(a,b)$ (outdated) then
    re-insert with the current value if positive; continue
  end if
  merge b into a; append (a,b) to seq; $k \leftarrow k+1$
  update the terms of Eq. (17) for a, b to get L_{cur}
  if $L_{cur} < L_{best} - \varepsilon$ then
    $L_{best} \leftarrow L_{cur}$; $k^* \leftarrow k$
  end if
  push $(\Delta Q(a,c), a, c)$ for neighbours c of a with $\Delta Q > 0$
end while
return the input partition after the first k^* merges of seq

codelength L. For an undirected graph, with module exit rates $q_i = \mathrm{cut}_i/2m$, module visit rates $p_i = \mathrm{vol}_i/2m$, node visit rates $p_\alpha = k_\alpha/2m$, $q = \sum_i q_i$ and $f(x) = x \log_2 x$,

$$L = f(q) - 2\sum_i f(q_i) + \sum_i f(q_i + p_i) - \sum_\alpha f(p_\alpha). \quad (17)$$

Lower is better. Merging two modules saves the announcements between them but lengthens the codebook of the merged module, so L falls only if the two modules are linked strongly enough; this is why its resolution limit depends on the links between modules and not on the size of the whole graph [17]. Our implementation reproduces the codelength of the Infomap package (to 10^{-15} for this form, and to four decimals for the regularised form below). On sparse graphs L can over-fit; the regularised codelength L_{reg} [18] is the map equation of G plus a complete graph of small weight $\ln N/N$ on every pair of nodes, which stops it from trusting thin local patterns.

M1: triangle-based similarity (replaces Jaccard in

Algorithm 4 SynwalkMoves(G, NI, L, S) (M6)

Input: graph G, importance NI, partition L, sweeps S
Output: partition L' (or L if the gate rejects the pass)

$L' \leftarrow L$; compute vol, cut of every module
for $s = 1$ to S do
  $changed \leftarrow 0$
  for all $i \in V$ in descending $NI(i)$ do
    $a \leftarrow L'(i)$; count links k_{ib} into each neighbouring module $b \neq a$
    $\Delta J_b \leftarrow j(a') + j(b') - j(a) - j(b)$ for each such b ▷ Eq. (22)
    $b^* \leftarrow \arg\max_b \Delta J_b$
    if $\Delta J_{b^*} > 0$ then
      move i to b^*; update vol, cut; $changed \leftarrow changed + 1$
    end if
  end for
  if $changed = 0$ then break
  end if
end for
if $L_{\mathrm{reg}}(L') < L_{\mathrm{reg}}(L)$ then
  return L'
else
  return L ▷ the gate rejects the pass
end if

Eq. (13)). With $c = |N(i) \cap N(j)|$ the number of triangles on edge (i, j),

$$\mathrm{Sim}(i,j) = \frac{c}{1+c} \quad (0 \to 0,\ 1 \to 0.5,\ 3 \to 0.75,\ 9 \to 0.9), \tag{18}$$

which depends on the triangle count and not on the sizes of the neighbourhoods. It costs a little on sparse graphs, where one shared neighbour is often chance but scores 0.5.

M2: best-iterate guard. Let Q_t be the modularity of the labels after Stage 2 round t ($t = 0$: the Stage 1 partition). The guard returns the labels of $\arg\max_t Q_t$ instead of the last round. It changes nothing unless a later round is worse.

M3: size-corrected gain. Stage 2 scores a candidate community by Eq. (16) with $\gamma = 1$. This is LPAm with importance-weighted edges; used alone it fragments graphs whose labels are coarse (Karate, Dolphin, Polbooks), hence M4.

M4: granularity rule (selector). Starting from the same Stage 1 partition we run the paper-style Stage 2 ($\gamma = 0$, with M1 and M2) giving $L^{(0)}$, and the corrected one ($\gamma = 1$, with M1) giving $L^{(1)}$. With $|L|$ the number of communities,

$$L = \begin{cases} L^{(1)} & \text{if } |L^{(1)}| \le \rho\,|L^{(0)}|, \\ L^{(0)} & \text{otherwise,} \end{cases} \qquad \rho = 1.5. \tag{19}$$

When both agree on the scale, $\gamma = 1$ only fine-tunes boundaries; when it shatters the graph (we measured $|L^{(1)}|/|L^{(0)}|$ of 2.0 for Karate and Dolphin, 2.3 for Polbooks, 5.0 for Polblogs, against ≤ 1.4 on Football, Citeseer, Cora and LFR) we are in the coarse-truth case and keep $L^{(0)}$. The test uses no labels. The value $\rho = 1.5$ was chosen by leave-one-dataset-out on our development graphs.

M5: map-equation merge. After Stage 2, adjacent communities a, b are merged greedily by the modularity gain

$$\Delta Q(a,b) = \frac{e_{ab}}{m} - \frac{d_a d_b}{2m^2}, \tag{20}$$

(e_{ab}: edges between them, d_a: volume), using a heap with lazy deletion (stale or outdated entries are skipped or refreshed when popped). The earlier version kept all merges if $\sum \Delta Q \ge 0.03$; on SNAP-DBLP it merged 9,803 communities into 807 and cut NMI from 0.7667 to 0.6197 because modularity favours merging at the wrong scale. We keep the merge order but evaluate the codelength L (Eq. (17)) after every merge (updating only the terms of the two communities, in $O(1)$) and return the prefix of merges at which L was lowest; if no prefix lowers L, nothing is merged. The map equation is used because its resolution limit depends on the links between modules and not on the size of the graph.

M6: boundary moves with a Synwalk score. The Synwalk objective [19] is a sum over modules of

$$j(\mathrm{vol}, \mathrm{cut}) = \frac{\mathrm{vol} - \mathrm{cut}}{2m} \ln \frac{1 - \mathrm{cut}/\mathrm{vol}}{\mathrm{vol}/2m} + \frac{\mathrm{cut}}{2m} \ln \frac{\mathrm{cut}/\mathrm{vol}}{1 - \mathrm{vol}/2m}, \tag{21}$$

which is the KL divergence between a module's stay/leave probabilities and those of a walker placed at random, weighted by the module's visit rate. Node i of degree k_i in module a, with k_{ia} links into a and k_{ib} into a neighbouring module b, moving $a \to b$ changes the modules to

$$\begin{aligned} (\mathrm{vol}'_a, \mathrm{cut}'_a) &= (\mathrm{vol}_a - k_i,\ \mathrm{cut}_a - (k_i - k_{ia}) + k_{ia}), \\ (\mathrm{vol}'_b, \mathrm{cut}'_b) &= (\mathrm{vol}_b + k_i,\ \mathrm{cut}_b + (k_i - k_{ib}) - k_{ib}), \end{aligned} \tag{22}$$

so a move is scored by recomputing two terms: $\Delta J = j(a') + j(b') - j(a) - j(b)$. Nodes are visited in decreasing NI and each moves to the neighbouring module with the largest positive ΔJ; sweeps repeat until no node moves (at most $S = 20$). There is no aggregation step, so communities are never joined wholesale. The whole pass is accepted only if the regularised codelength L_{reg} also falls; otherwise all moves are discarded. The gate exists because the plain map equation over-fits small graphs: one move costs 0.16 NMI on Karate. It was designed after seeing that failure, so it is a post-hoc choice.

Stage 3. Unchanged (Eq. (15)), applied to the final partition.

Complexity. Per Stage 2 round, similarities are computed once per edge, and one round scans each edge a constant number of times; the granularity rule runs Stage 2 twice. The merge pass is a heap-driven procedure whose cost is dominated by heap operations over adjacent community pairs; the moves cost $O(S \cdot m)$ at most, since a move scores one candidate per neighbouring module. We have not proved tighter bounds; Table ?? gives measured times.

IV. Experimental Results

A. Setup and Protocol

Methods compared. Fastgreedy [10], LPA [5], Louvain [11] and Walktrap [13] (all through the igraph C library), the baseline TPOC-LPA as we re-implemented it (TPOC†), and our pipeline (M1–M6). Infomap [14] is not in [1]; we add it because M5 and M6 borrow from it, so it is the strongest reference. For overlap we compare with SLPA [22] (T=100), implemented by us, with r tuned per graph as the paper tunes it. NSA, SimCMR, WLC, UMSTMO, GREESE and MDPA have no public code that we could find, and we did not re-implement them from a paragraph of description; their published numbers are quoted in Table XIII and marked as such.

Conventions in the tables. FG = Fastgreedy, Louv. = Louvain, Walk. = Walktrap, Info.* = Infomap (extra baseline) and TPOC† = our re-implementation of TPOC-LPA. Stochastic methods are the mean of 10 runs (3 on the SNAP and LFR graphs); the best value of each row is in bold; "Mean" and "Mean rank" are taken over the rows shown. SNAP* rows are our own induced subgraphs, with the first listed community as truth. Table VI uses the settings of Table III (mean of 5 graphs per row, with $c_{\min}$=15 instead of 10 for n=1000), and Table VIII averages om = 2 to 8 over two generator seeds, with r of SLPA tuned per graph by LFK-NMI, which favours SLPA.

Protocol. We kept development graphs (Karate, Dolphin, Football, Polbooks, Polblogs, email-Eu, Cora, Citeseer, Amazon Photo, LFR n=1000 seeds 1–5) apart from held-out graphs (72 further LFR variants, Amazon Computers, Coauthor-CS, Coauthor-Physics) and fresh ones used once. A change became a candidate default only if no held-out real graph lost more than 0.005 NMI. Stochastic baselines are averaged over 10 runs (3 on SNAP and LFR); we also computed their best run, which leaves the conclusions unchanged (mean rank on real graphs 2.4 for our pipeline; LFR mean 0.9941 against 0.9931 for Infomap). Our own pipeline and TPOC† are deterministic.

NMI normalisation. The normalisation used in [1] is not clear to us: for the same Fastgreedy partition of Karate it reports 0.7069 while we obtain 0.5646, and no standard variant reproduces its value. We use one metric implementation for all our columns, so only differences within our tables are meaningful.

B. Real Networks

Table V gives the main result. The pipeline has the best mean rank (2.00 against 4.09–4.73 for every other column) and the highest mean NMI (0.7031 against 0.6585 for Infomap). Part of this margin comes from email-Eu, where the baseline collapses (0.0000) and our pipeline reaches 0.6677, above Louvain (0.5806) and Infomap (0.5873). On the three graphs shared with [1] (Karate, Dolphin, Football) it scores 0.8372, 0.9495 and 0.9242, above Infomap (0.5805, 0.8993, 0.8683). On SNAP-DBLP it is marginally ahead (0.7676 against 0.7657 for LPA and 0.7631 for Infomap).

It is not best everywhere. On Cora, Fastgreedy is higher (0.4647 against 0.4192); on SNAP-Amazon, Louvain is (0.9825 against 0.9731); and on SNAP-Youtube Infomap (0.7486) and Walktrap (0.7222) are far ahead of our 0.5745, which is itself better than the baseline's 0.5195.

C. LFR Benchmarks and Ablation

Table VI shows that the mean NMI of our pipeline over the six settings is 0.9941, against 0.9905 for Infomap and 0.9805 for TPOC†. At μ=0.6 the mean is 0.9883 against 0.9827 for Infomap and 0.9629 for TPOC†. Infomap is slightly higher on n=5000 and n=10000 at μ=0.6 (0.9945 against 0.9940 and 0.9900 against 0.9895), which is within seed noise. The published baseline reports 0.9869 on its own n=1000, μ=0.6 graphs; on our five graphs of that setting TPOC† reaches 0.9317 and our pipeline 0.9815. Table VII shows where the gain comes from; its rows are development runs (15 graphs for each LFR column, one graph for email-Eu) with the merge pass M5 switched off.

TABLE VII
Effect of the modifications on NMI.

Configuration	μ=0.6	μ=0.7	email-Eu
TPOC-LPA (re-implemented baseline)	0.9311	0.0513	0.0000
+ M1–M4 (motif, guard, size-corr., granularity)	0.9637	0.8534	0.6381
+ M6, map-equation move score	0.9776	0.8759	0.6554
+ M6, Synwalk move score	0.9830	0.8917	0.6717

On the 72 held-out synthetic graphs, the configuration with M1–M4 averaged 0.7814 and the final pipeline with M5 and M6 averaged 0.8069, better on 65, equal on 7 and worse on 3 graphs (worst −0.010).

D. Overlapping Communities

Table VIII reports overlap quality. With the paper's own WAS Stage 3 on top of our Stage 2, the pipeline has the highest mean LFK-NMI (0.7673 against 0.7578 for tuned SLPA and 0.7600 for TPOC†), and it wins R1–R4. It loses R5 and R6 (the 10,000-node families) to SLPA, and on the Omega mean SLPA is ahead (0.7571 against 0.7418).

TABLE V
NMI of different algorithms on real networks.

Network	FG	LPA	Louv.	Walk.	Info.*	TPOC†	SN-LPA
Karate	0.5646	0.5254	0.5244	0.4899	0.5805	0.8372	0.8372
Dolphin	0.7026	0.8139	0.8297	0.6324	0.8993	0.9053	0.9495
Football	0.6977	0.8666	0.8820	0.8874	0.8683	0.9095	0.9242
Polbooks	0.5308	0.5495	0.5499	0.5427	0.5437	0.5156	0.5672
Polblogs	0.6541	0.7094	0.6379	0.6441	0.5270	0.7094	0.7107
email-Eu	0.4417	0.0122	0.5806	0.5685	0.5873	0.0000	0.6677
Cora	0.4647	0.4328	0.4631	0.4427	0.4166	0.4192	0.4192
Citeseer	0.3421	0.3405	0.3300	0.3541	0.3386	0.3465	0.3438
SNAP-Amazon*	0.9820	0.9610	0.9825	0.9774	0.9710	0.9586	0.9731
SNAP-Youtube*	0.5212	0.5040	0.5410	0.7222	0.7486	0.5195	0.5745
SNAP-DBLP*	0.6040	0.7657	0.6068	0.7497	0.7631	0.7626	0.7676
Mean	0.5914	0.5892	0.6298	0.6374	0.6585	0.6258	0.7031
Mean rank	4.64	4.73	4.09	4.09	4.18	4.18	2.00

TABLE VI
NMI of different algorithms on synthetic networks.

Network	FG	LPA	Louv.	Walk.	Info.*	TPOC†	SN-LPA
N1 (n=1000, μ=0.1)	0.8098	0.9880	0.9872	0.9969	0.9951	0.9942	0.9995
N2 (n=1000, μ=0.6)	0.4749	0.4245	0.9184	0.9560	0.9637	0.9317	0.9815
N3 (n=5000, μ=0.1)	0.9013	1.0000	0.9956	1.0000	1.0000	1.0000	1.0000
N4 (n=5000, μ=0.6)	0.3879	0.9828	0.9110	0.9598	0.9945	0.9812	0.9940
N5 (n=10000, μ=0.1)	0.8921	1.0000	0.9864	1.0000	0.9999	1.0000	1.0000
N6 (n=10000, μ=0.6)	0.3825	0.9773	0.9101	0.9178	0.9900	0.9758	0.9895
Mean	0.6414	0.8954	0.9515	0.9718	0.9905	0.9805	0.9941
Mean rank	6.83	3.33	5.83	2.83	2.17	3.00	1.33

TABLE VIII
Average LFK-NMI, $\mathrm{NMI}_{\max}$ and Omega values of different algorithms on overlapping networks, *om* from 2 to 8.

	LFK-NMI			$\mathrm{NMI}_{\max}$			Omega		
Family	SLPA	TPOC†	SN-LPA	SLPA	TPOC†	SN-LPA	SLPA	TPOC†	SN-LPA
R1 (n=10^3, μ=0.1)	0.7914	0.7949	0.8240	0.7602	0.7480	0.8002	0.8019	0.7010	0.8125
R2 (n=10^3, μ=0.3)	0.6785	0.6831	0.7144	0.6549	0.6344	0.6851	0.6706	0.5127	0.6693
R3 (n=$5{\cdot}10^3$, μ=0.1)	0.8474	0.8689	0.8759	0.8137	0.8507	0.8597	0.8384	0.8642	0.8808
R4 (n=$5{\cdot}10^3$, μ=0.3)	0.7210	0.7409	0.7520	0.7077	0.7042	0.7330	0.7218	0.7093	0.7491
R5 (n=10^4, μ=0.1)	0.7726	0.7696	0.7552	0.7294	0.6972	0.7142	0.7804	0.6147	0.7273
R6 (n=10^4, μ=0.3)	0.7362	0.7028	0.6822	0.7029	0.5900	0.6275	0.7294	0.4595	0.6119
Mean	0.7578	0.7600	0.7673	0.7281	0.7041	0.7366	0.7571	0.6436	0.7418

E. Extended Modularity (EQ)

TABLE IX
EQ of different algorithms on real networks.

Network	SLPA	TPOC†	SN-LPA
Karate (n=34)	0.3717	0.3516	0.3516
Dolphin (n=62)	0.5199	0.4778	0.4828
Jazz (n=198)	0.2820	0.2848	0.2871
Lesmis (n=77)	0.3419	0.4788	0.4989
Cora (n=2708)	0.7019	0.6623	0.6402
Power (n=4941)	0.6520	0.7479	0.8030
Facebook (n=4039)	0.8015	0.7926	0.8090
CA-GrQc (n=5241)	0.7866	0.6966	0.7274
LastFM (n=7624)	0.7562	0.7510	0.7594
CA-HepTh (n=9875)	0.6703	0.5212	0.5882
PGP (n=10680)	0.7851	0.7030	0.7421
Deezer (n=28281)	0.5461	0.4321	0.4145
Mean	0.6013	0.5750	0.5920
Mean rank	1.67	2.50	1.75

Table IX: our pipeline is best on 5 of 12 networks (Jazz, Lesmis, Power, Facebook, LastFM), and SLPA on the other 7; for SLPA the threshold r was scanned from 0.05 to 0.5 and the best EQ kept, so it is tuned on EQ itself. The mean EQ is 0.5920 for ours against 0.6013 for SLPA and 0.5750 for TPOC†.

F. Latency and Scalability

We repeat the scalability experiment of Sec. 5.3 of [1]: overlapping LFR graphs with average degree 5, maximum degree 10, μ=0.1, community sizes 10 to 20, *om*=2, *on*=10 and N from 400 to 25,600 nodes, doubling each time. Every algorithm runs on the same graph, one after the other, on an otherwise idle machine; the time is the wall-clock time of the call (including, for the igraph methods, the conversion of the graph), and we report the median of 3 runs. Table X and Fig. 3 show the result; the last column is the slope of a least-squares line on the logarithms of N

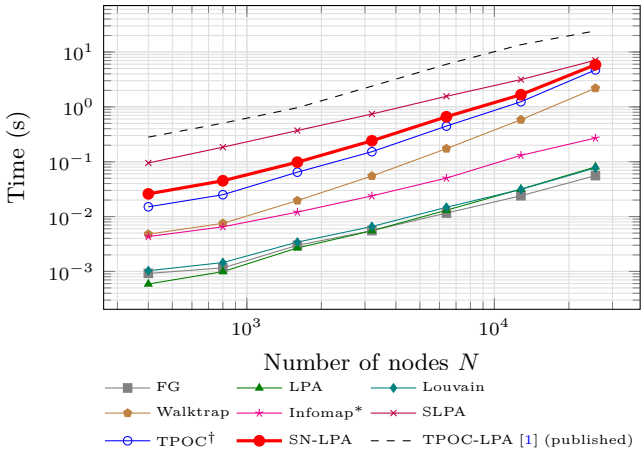


Fig. 3. Running time versus the number of nodes.

and time, that is the empirical growth exponent (1 means linear growth).

At $N = 25,600$ our pipeline needs 5.87 s, which is 24% more than our re-implementation of the baseline (4.72 s), about 22 times the time of Infomap (0.27 s), about 76 times that of Louvain (0.077 s) and 2.7 times that of Walktrap (2.19 s); it is faster than SLPA (7.09 s). LPA, Louvain, Fastgreedy, Walktrap and Infomap are implemented in C or C++, whereas TPOC-LPA, our pipeline and SLPA are pure Python, so most of the gap to the first group is the implementation language and not the algorithm; we did not optimise our code. The growth exponent of our pipeline is 1.31, against 1.39 for our baseline re-implementation and 1.12 for the times published in [1], whose claim is that the algorithm is close to linear. Our baseline therefore does not reproduce a linear growth; the published times come from a different implementation and machine, so we use them only for their trend.

Table XI explains where the time goes. Stage 1 takes 62% of the total at $N = 25,600$ and only 9% at $N = 400$. In our literal reading of Stage 1 the tightness–sparsity value of the growing community is recomputed from its members for every candidate neighbour, so a cost that depends on the community size appears inside the loop and makes this stage grow faster than linearly. The modifications themselves (the second Stage 2 run of the granularity rule, the merge pass and the moves) add 0.90 s at the largest size, and none of them dominates. An incremental update of the community's tightness would reduce the Stage 1 cost; we have not done this.

Table XII gives the time on the real networks, with the same protocol (3 runs, median; one run for the cases that take longer than 20 s). Our pipeline needs about the same time as the baseline re-implementation, from 1 ms on Karate to 83 s on SNAP-DBLP (93,432 nodes), where the baseline needs 77 s. The C implementations of LPA, Louvain and Infomap need at most 2.1 s on any of these graphs and Fastgreedy at most 12 s; Walktrap needs 76 s on SNAP-DBLP and 243 s on SNAP-Youtube. On the largest graphs a run of our pipeline is therefore of the order of a minute, which is acceptable for offline analysis but far from the speed of the C methods.

G. Published Numbers of the Baselines Without Public Code

TABLE XIII
NMI as published in [1] (not re-run).

Network	FG	LPA	Louv.	Walk.	NSA	SimCMR	TPOC-LPA
Karate	0.7069	0.8372	0.7236	0.5467	0.7112	0.7110	1.0000
Dolphin	0.7377	0.8929	0.7830	0.6329	0.8929	0.8313	0.9053
Football	0.7569	0.9063	0.8909	0.8879	0.8889	0.8973	0.9146
N1	0.9480	1.0000	0.9977	1.0000	0.9906	0.9977	1.0000
N2	0.4659	0.9388	0.9233	0.9591	0.5157	0.9158	0.9869
N3	0.9212	1.0000	0.9970	1.0000	0.9419	0.9965	1.0000
N4	0.3515	0.9899	0.9234	0.9397	0.4179	0.9146	0.9974
N5	0.9109	1.0000	0.9910	1.0000	0.9114	0.9905	1.0000
N6	0.3786	0.9925	0.9169	0.7070	0.2087	0.8861	0.9935

Table XIII lists the values published in [1], including the two baselines (NSA and SimCMR) for which we found no implementation. They are not comparable with Tables V and VI, because the NMI normalisation and the graph instances differ.

V. Discussion

A. What Drives the Gains

The gains come from different modifications on different graphs (Table VII). M1–M4 remove the collapse: email-Eu goes from 0.0000 to 0.6381 and LFR μ=0.7 from 0.0513 to 0.8534. M6 adds a second, smaller gain on the high-mixing LFR graphs and on email-Eu (LFR μ=0.6: 0.9637 $\rightarrow$ 0.9830), consistent with its role of repairing boundary nodes, which is the error that remains after Stage 2. The Synwalk score beats the map-equation score on these graphs (0.9830 against 0.9776 at μ=0.6, 0.8917 against 0.8759 at μ=0.7), in line with the reported advantage of Synwalk at high mixing, but it is slightly worse on coarse and sparse real graphs. M5 matters on large sparse graphs: the modularity merge reduced NMI on SNAP-DBLP by 0.147 and on fresh sparse LFR graphs by 0.12–0.23, while the map-equation stop changed it by between -0.008 and $+0.005$.

B. Where the Pipeline Loses

SNAP-Youtube. The ground truth has 4,447 communities. The two methods that return many communities are far ahead: Infomap returns about 2,345 (NMI 0.7486) and Walktrap 4,567 (0.7222). Every method that returns 955 to 1,389 communities scores between 0.50 and 0.57: Louvain about 955 (0.5410), LPA about 1,202 (0.5040), our pipeline 1,209 (0.5745), Fastgreedy 1,262 (0.5212) and TPOC† 1,389 (0.5195). Our pipeline returns fewer communities than TPOC†, so the merge and move steps reduce the count further. This is consistent with Youtube having many small communities that a coarser partition

TABLE X
Execution time (s) of different algorithms for different N.

Algorithm	400	800	1,600	3,200	6,400	12,800	25,600	Slope
FG	0.001	0.001	0.003	0.006	0.012	0.024	0.056	1.02
LPA	0.001	0.001	0.003	0.006	0.013	0.032	0.080	1.20
Louvain	0.001	0.001	0.003	0.007	0.015	0.031	0.077	1.06
Walktrap	0.005	0.008	0.020	0.055	0.173	0.583	2.19	1.51
Infomap*	0.004	0.006	0.012	0.024	0.050	0.131	0.270	1.02
SLPA	0.095	0.184	0.370	0.741	1.56	3.14	7.09	1.03
TPOC†	0.015	0.025	0.064	0.152	0.446	1.24	4.72	1.39
SN-LPA	0.026	0.045	0.098	0.241	0.661	1.67	5.87	1.31
TPOC-LPA [1] (published)	0.281	0.505	0.965	2.39	5.97	13.61	24.33	1.12

TABLE XI
Execution time (s) of each stage of the proposed pipeline.

N	NI	Stage 1	Stage 2	M4	M5+M6	Stage 3	Total
400	0.001	0.002	0.013	0.005	0.003	0.001	0.025
800	0.002	0.006	0.019	0.008	0.009	0.001	0.045
1,600	0.004	0.019	0.038	0.017	0.017	0.002	0.098
3,200	0.008	0.062	0.092	0.039	0.036	0.004	0.240
6,400	0.016	0.240	0.214	0.089	0.089	0.009	0.655
12,800	0.033	0.914	0.373	0.177	0.188	0.019	1.71
25,600	0.071	3.66	1.19	0.545	0.357	0.039	5.92

TABLE XII
Execution time (s) of different algorithms on real networks.

Network	FG	LPA	Louv.	Walk.	Info.*	TPOC†	SN-LPA
Karate	<0.01	<0.01	<0.01	<0.01	<0.01	<0.01	<0.01
Dolphin	<0.01	<0.01	<0.01	<0.01	<0.01	<0.01	<0.01
Football	<0.01	<0.01	<0.01	<0.01	<0.01	0.01	0.01
Polblogs	0.03	0.01	0.01	0.11	0.03	1.41	1.33
email-Eu	0.02	0.01	0.01	0.08	0.03	0.91	0.95
Cora	0.01	0.01	0.01	0.08	0.03	0.17	0.24
Citeseer	<0.01	<0.01	<0.01	0.03	0.02	0.11	0.16
SNAP-Amazon	0.08	0.04	0.04	0.82	0.15	2.56	3.05
SNAP-Youtube	7.05	0.19	0.19	243	0.83	70.62	65.92
SNAP-DBLP	12.07	0.74	0.42	75.72	2.11	77.15	82.69

combines, but we have not tested it; it remains an open problem.

Sparse graphs with coarse labels. On Cora and Citeseer the pipeline is no better than the baseline (0.4192 against 0.4192; 0.3438 against 0.3465), and the moves cost up to 0.008 on sparse real graphs. On graphs whose labels are coarse (Coauthor-Physics, Coauthor-CS, Cora) a modularity merge scores higher than the map-equation stop (Physics 0.4324 against 0.2790) because the resolution limit of modularity happens to match the coarse labels; no label-free criterion that we tried could tell this case from fine-grained truth, and every candidate (a map-equation guard, a Synwalk guard, relative-codelength vetoes) failed on some graph.

Karate. Our NMI of 0.8372 stays below the 1.0000 reported in [1]: node 8 has three links to the other faction and two to its own, so any structure-only rule assigns it the other way, and the plain LPA baseline in the same paper also scores 0.8372. We also found that networkx's Karate graph carries edge weights; on the unweighted graph our partition has $Q = 0.3715$, which equals the Q reported in [1], whereas the true two-faction split has $Q = 0.3582$. This suggests that the reported Q and NMI do not come from the same partition, but we cannot prove it.

C. Overlap

The overlap result is the weakest part. Our re-implemented baseline scores below the values published for TPOC-LPA on the same families (for instance 0.7949 against 0.8358 for R1 under LFK-NMI), and we could not close that gap. We decomposed the loss on R1 and R2: given our Stage 2 partition, a perfect Stage 3 would reach LFK-NMI 0.84–0.98, so most of the loss is in Stage 3. But the missing memberships are mostly nodes with a single link into the other community, and such a link is a true membership only 7–11% of the time, so no local rule can find them; a rule based on triangles closed inside the candidate community had precision 0.92–0.99 on R1 (three or more embedded links) but found too few memberships and added nothing to WAS. The EQ result is mixed: tuned SLPA is competitive on EQ (and its r is tuned on EQ itself), and we did not investigate the gap further.

D. Failed Ideas

These were tried and dropped; they remain in the code as "explored, not adopted".

- LLM correction of Stage 2 with local 3–8B models. Oracle headroom exists (Football +0.059), but a hard override with the measured accuracy of 0.817 gave NMI 0.7815 against 0.9095 structure-only.
- Map-equation guard in place of modularity (M2): email-Eu fell from 0.64 to 0.00, because the map equation prefers one module when structure is weak.
- Persona (ego-splitting) Stage 3: it matched the published overlap numbers on our design graphs and helped on held-out synthetic ones, but lost on real graphs (SNAP-Amazon −0.009, SNAP-DBLP −0.015 $\mathrm{NMI}_{\max}$ against the best WAS).
- Near-tie and embedded-link overlap rules: they help SNAP-DBLP only, or add nothing once WAS is applied.
- Selecting α by codelength, link-majority polishing and consensus clustering did not pass the held-out test.

E. Threats to Validity

- Post-hoc choices. $\rho = 1.5$ (picked on development data), the regularised gate in M6, $s = 3$ for personas, and $\beta = 0.8$ taken from the paper. A choice made after seeing development data may not transfer.
- No universal improvement. All modifications remain opt-in; Coauthor-CS and Coauthor-Physics fail our adoption rule.
- Different graphs and metric. SNAP subgraphs and the NMI normalisation differ from those of [1]; cross-paper numbers are indicative only, and we could not reproduce its overlapping-LFR and EQ values for our baseline implementation.
- Baselines. Six baselines of the paper are not re-run, Leiden is not included, and SLPA is tuned with the labels, which favours it. The overlap result rests on two generator seeds.
- Novelty. Each component exists in prior work; the contribution is the analysis of where TPOC-LPA fails, the combination and the gates, and the comparison.

VI. Conclusion

We re-implemented TPOC-LPA, identified an unnormalised size effect in its Stage 2 that explains its collapse on dense graphs and under-merging on sparse ones, and added six optional modifications that keep the published behaviour as the default: a triangle similarity, a best-iterate guard, a size-corrected gain with a label-free granularity rule, a merge pass stopped by the map equation, and boundary moves scored by the Synwalk objective and accepted through a regularised gate. Compared with baselines re-run on identical graphs, the pipeline has the best mean rank on real networks and the highest mean NMI on LFR, repairs the collapse on dense graphs, and is competitive on overlap, but it is far behind Infomap on SNAP-Youtube, behind SLPA on the largest overlapping benchmarks and below the published EQ values; it is about 25% slower than our baseline re-implementation, and both are far slower than the C implementations of the other methods.

Future work. (i) Diagnose the Youtube gap, starting from the number of communities. (ii) Calibrate the overlap rate without labels. (iii) Add Leiden and, if code is released, the six baselines we could not run. (iv) Test the pipeline on more fresh graphs before proposing any of the changes as defaults.

References

[1] Z. Kong, Y. Huo, Y. Wang and Y. Guo, "Three-stage overlapping community detection algorithm based on label propagation," Expert Syst. Appl., vol. 317, art. 131873, 2026. https://doi.org/10.1016/j.eswa.2026.131873

[2] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," Proc. Natl. Acad. Sci. USA, vol. 99, no. 12, pp. 7821–7826, 2002. https://doi.org/10.1073/pnas.122653799

[3] S. Fortunato, "Community detection in graphs," Phys. Rep., vol. 486, nos. 3–5, pp. 75–174, 2010. https://doi.org/10.1016/j.physrep.2009.11.002

[4] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Phys. Rev. E, vol. 69, 026113, 2004. https://doi.org/10.1103/physreve.69.026113

[5] U. N. Raghavan, R. Albert and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," Phys. Rev. E, vol. 76, 036106, 2007. https://doi.org/10.1103/physreve.76.036106

[6] I. X. Y. Leung, P. Hui, P. Liò and J. Crowcroft, "Towards real-time community detection in large networks," Phys. Rev. E, vol. 79, 066107, 2009. https://doi.org/10.1103/physreve.79.066107

[7] V. A. Traag and L. Šubelj, "Large network community detection by fast label propagation," Sci. Rep., vol. 13, 2701, 2023. https://doi.org/10.1038/s41598-023-29610-z

[8] M. J. Barber and J. W. Clark, "Detecting network communities by propagating labels under constraints," Phys. Rev. E, vol. 80, 026129, 2009. https://doi.org/10.1103/physreve.80.026129

[9] X. Liu and T. Murata, "Advanced modularity-specialized label propagation algorithm for detecting communities in networks," Physica A, vol. 389, no. 7, pp. 1493–1500, 2010. https://doi.org/10.1016/j.physa.2009.12.019

[10] A. Clauset, M. E. J. Newman and C. Moore, "Finding community structure in very large networks," Phys. Rev. E, vol. 70, 066111, 2004. https://doi.org/10.1103/physreve.70.066111

[11] V. D. Blondel, J.-L. Guillaume, R. Lambiotte and E. Lefebvre, "Fast unfolding of communities in large networks," J. Stat. Mech., P10008, 2008. https://doi.org/10.1088/1742-5468/2008/10/p10008

[12] V. A. Traag, L. Waltman and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," Sci. Rep., vol. 9, 5233, 2019. https://doi.org/10.1038/s41598-019-41695-z

[13] P. Pons and M. Latapy, "Computing communities in large networks using random walks," J. Graph Algorithms Appl., vol. 10, no. 2, pp. 191–218, 2006. https://doi.org/10.7155/jgaa.00124

[14] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proc. Natl. Acad. Sci. USA, vol. 105, no. 4, pp. 1118–1123, 2008. https://doi.org/10.1073/pnas.0706851105

[15] A. Lancichinetti and S. Fortunato, "Community detection algorithms: a comparative analysis," Phys. Rev. E, vol. 80, 056117, 2009. https://doi.org/10.1103/physreve.80.056117

[16] S. Fortunato and M. Barthélemy, "Resolution limit in community detection," Proc. Natl. Acad. Sci. USA, vol. 104, no. 1, pp. 36–41, 2007. https://doi.org/10.1073/pnas.0605965104

[17] T. Kawamoto and M. Rosvall, "Estimating the resolution limit of the map equation in community detection," Phys. Rev. E, vol. 91, 012809, 2015. https://doi.org/10.1103/physreve.91.012809

[18] J. Smiljanić, D. Edler and M. Rosvall, "Mapping flows on sparse networks with missing links," Phys. Rev. E, vol. 102, 012302, 2020. https://doi.org/10.1103/physreve.102.012302

[19] C. Toth, D. Helic and B. C. Geiger, "Synwalk – community detection via random walk modelling," Data Min. Knowl. Discov., vol. 36, no. 2, pp. 739–780, 2022. https://doi.org/10.1007/s10618-021-00809-w

[20] P.-Z. Li, L. Huang, C.-D. Wang, J.-H. Lai and D. Huang, "Community detection by motif-aware label propagation," ACM Trans. Knowl. Discov. Data, vol. 14, no. 2, 2020. https://doi.org/10.1145/3378537

[21] G. Palla, I. Derényi, I. Farkas and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," Nature, vol. 435, pp. 814–818, 2005. https://doi.org/10.1038/nature03607

[22] J. Xie, B. K. Szymanski and X. Liu, "SLPA: Uncovering overlapping communities in social networks via a speaker-listener interaction dynamic process," in Proc. IEEE ICDM Workshops, 2011. https://doi.org/10.1109/icdmw.2011.154

[23] S. Gregory, "Finding overlapping communities in networks by label propagation," New J. Phys., vol. 12, 103018, 2010. https://doi.org/10.1088/1367-2630/12/10/103018

[24] A. Epasto, S. Lattanzi and R. Paes Leme, "Ego-splitting framework: from non-overlapping to overlapping clusters," in Proc. ACM KDD, 2017. https://doi.org/10.1145/3097983.3098054

[25] A. Lancichinetti, S. Fortunato and F. Radicchi, "Benchmark graphs for testing community detection algorithms," Phys. Rev. E, vol. 78, 046110, 2008. https://doi.org/10.1103/physreve.78.046110

[26] A. Lancichinetti, S. Fortunato and J. Kertész, "Detecting the overlapping and hierarchical community structure in complex networks," New J. Phys., vol. 11, 033015, 2009. https://doi.org/10.1088/1367-2630/11/3/033015

[27] L. M. Collins and C. W. Dent, "Omega: a general formulation of the Rand index of cluster recovery suitable for non-disjoint solutions," Multivariate Behav. Res., vol. 23, no. 2, pp. 231–242, 1988. https://doi.org/10.1207/s15327906mbr2302_6

[28] H. Shen, X. Cheng, K. Cai and M.-B. Hu, "Detect overlapping and hierarchical community structure in networks," Physica A, vol. 388, no. 8, pp. 1706–1712, 2009. https://doi.org/10.1016/j.physa.2008.12.021

[29] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," in Proc. IEEE ICDM, 2012. https://doi.org/10.1109/icdm.2012.138

[30] W. W. Zachary, "An information flow model for conflict and fission in small groups," J. Anthropol. Res., vol. 33, no. 4, pp. 452–473, 1977. https://doi.org/10.1086/jar.33.4.3629752

[31] D. Lusseau et al., "The bottlenose dolphin community of Doubtful Sound features a large proportion of long-lasting associations," Behav. Ecol. Sociobiol., vol. 54, pp. 396–405, 2003. https://doi.org/10.1007/s00265-003-0651-y

[32] L. A. Adamic and N. Glance, "The political blogosphere and the 2004 U.S. election: divided they blog," in Proc. 3rd Int. Workshop on Link Discovery, 2005. https://doi.org/10.1145/1134271.1134277

[33] H. Yin, A. R. Benson, J. Leskovec and D. F. Gleich, "Local higher-order graph clustering," in Proc. ACM KDD, 2017. https://doi.org/10.1145/3097983.3098069

[34] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher and T. Eliassi-Rad, "Collective classification in network data," AI Mag., vol. 29, no. 3, pp. 93–106, 2008. https://doi.org/10.1609/aimag.v29i3.2157